

Tipos de Datos

...

Programación
2025

Resolución de problemas (de nuevo)

Análisis



Entradas y salidas

Diseño



Pasos y Pseudocódigo

Implementación



Código

Depuración



Corrección de errores

Python

- Un lenguaje de programación permite dar **instrucciones** a un **sistema informático**.
- Python es un lenguaje
 - de uso general
 - multiplataforma
 - de (muy) alto nivel.
 - interpretado
 - orientado a objetos
 - de tipado dinámico.
 - con una extensa librería de funciones.
 - con una comunidad activa.

Ejemplo de Programa en Python

```
# Código para calcular una hipotenusa
```

```
cat_menor = 3
```

```
cat_mayor = 4
```

```
hip = (cat_menor ** 2 + cat_mayor ** 2)
```

```
hip = hip ** 0.5 # “**” significa “elevado a”
```

```
print(hip)
```

Ejemplo de Programa en Python

```
# Código para calcular una hipotenusa
```

```
cat_menor = 3
```

```
cat_mayor = 4
```

Operadores



```
hip = (cat_menor ** 2 + cat_mayor ** 2)
```

```
hip = hip ** 0.5 # “**” significa “elevado a”
```

```
print(hip)
```

Ejemplo de Programa en Python

Código para calcular una hipotenusa

Variables



```
cat_menor = 3  
cat_mayor = 4
```

Operadores



```
hip = (cat_menor ** 2 + cat_mayor ** 2)  
hip = hip ** 0.5 # "**" significa "elevado a"
```

```
print(hip)
```

Ejemplo de Programa en Python

Código para calcular una hipotenusa

Variables

cat_menor = 3
cat_mayor = 4

Operadores

hip = (cat_menor ** 2 + cat_mayor ** 2)
hip = hip ** 0.5 # "**" significa "elevado a"

print(hip)

Funciones

Ejemplo de Programa en Python

Código para calcular una hipotenusa

cat_menor = 3

cat_mayor = 4

hip = (cat_menor ** 2 + cat_mayor ** 2)

hip = hip ** 0.5 # "**" significa "elevado a"

print(hip)

Variables

Operadores

Comentarios

Funciones

Variables

- Permiten almacenar datos en memoria.
- Para asignar un valor a una variable se usa el signo igual (=).
- Si los datos no se guardan en una variable, se pierden luego de ser calculados

```
cat_menor = 3  
cat_mayor = 4
```

```
(cat_menor ** 2 + cat_mayor ** 2) ** 0.5  
# Se hace el cálculo, pero no se tiene acceso al resultado!!!
```

Variables

- Los nombres de las variables (y de las funciones) deben cumplir ciertas reglas:
 - Empezar con una letra.
 - El resto del nombre puede contener solo letras, números y guiones bajos (_)
 - Se distingue entre mayúsculas y minúsculas (“Variable” y “variable” son dos nombres distintos!)

Palabras reservadas

and

continue

except

global

lambda

raise

yield

as

def

exec

if

not

return

assert

del

finally

import

or

try

break

elif

for

in

pass

while

class

else

from

is

with

False

None

True

nonlocal

Tipos de Datos

- Primitivos:
 - Enteros (int)
 - Reales (float)
 - Cadenas de caracteres (strings)
 - Verdadero / falso (boolean)

- Abstractos:
 - Tuplas y listas
 - Diccionarios
 - Grafos
 - Árboles
 - ...

*Algoritmos y Estructuras
de Datos!!*

Tipos de Datos Numéricos

- Existen tres tipos de datos numéricos nativos en Python: **enteros**, **flotantes** y **complejos**.

- Los **enteros** pueden ser tanto positivos como negativos:

- 5

- -5

- Los **flotantes** son números que admiten decimales:

- 5.0

- -1.9

- 6.022E23

- 0.0000038

- No existe un límite para el largo de los números (pero para números muy grandes conviene usar notación científica).

Cadenas de caracteres

- Permiten almacenar información en formato de texto.
- Un **string** puede almacenar cualquier tipo de carácter del código ASCII, como letras, números, signos de puntuación y espacios .
- Para definir un string se pueden utilizar tanto comillas simples (' ') como dobles (" ").
 - "Este es un string"
 - 'Este es otro string'
 - "223344"
 - "...!!"

Codigo ASCII

[illegible]

Caracteres especiales comunes

- Para ingresar ciertos elementos en un string es necesario utilizar una sintaxis especial.
- Todos estos caracteres comienzan con una barra invertida (“\”)

Caracter	Resultado
\'	Comilla simple dentro del string
\"	Comilla doble dentro del string
\n	Fin de linea
\t	tabulacion

- El string *I'm learning so much today* se podría almacenar como:
 - `string1 = 'I\'m learning so much today'`
 - `string2 = "I'm learning so much today"`

Tipos de Datos Lógicos

- Los booleanos admiten dos valores: verdadero (**True**) y falso (**False**)
- Son muy utilizados para evaluar alternativas (“si, sino”).
 - ¿ $32 > 12$? $\Rightarrow$ **True**
 - ¿ “arbol” > “bici” ? $\Rightarrow$ **False**
 - ¿ “vaca” > “ñandu” ? $\Rightarrow$ **False**

Constantes

- Es un dato que no se modifica en toda la ejecución del programa.
- En Python no existe el tipo de dato “constante”, sino que se definen variables (usualmente, nombres en mayúsculas) que luego no se modifican.
- Algunos ejemplos de constantes muy utilizadas:
 - $\pi = 3.1415$
 - $G = 9.81$
 - $E = 2.7182$

Comentarios

- Son líneas en el código que no son leídas por el intérprete.
- Comienzan con un numeral (#).
- Si se desean comentar más de una línea, se pueden utilizar 3 comillas dobles (" " ")

```
# Esto es una línea con comentarios
var1 = "Esto es una variable del tipo string"
# Esto es otro comentario
var2 = 7 # Los comentarios pueden ir al final de una línea
" " "

Acá van
varias
líneas de comentarios
" " "
```

Operaciones entre Números

Operaciones aritméticas entre números (int y float):

Operación	Significado
$x + y$	Suma
$x - y$	Resta
$x * y$	Multiplicación
x / y	División
$x ** b$	Exponencial ("x" elevado a la "y")
$x \% y$	Módulo (resto de la división)
$x // y$	División entera

Ejemplos:

$$7 / 3 = 2.33$$

$$7 // 3 = 2$$

$$7 \% 3 = 1$$

Operaciones entre Números

Asignaciones entre números (int y float)

Operación	Equivalente a	Significado
<code>x += y</code>	<code>x = x + y</code>	Asigna a "x" la suma entre "x" y "y"
<code>x -= y</code>	<code>x = x - y</code>	Asigna a "x" la resta entre "x" y "y"

Ejemplo:

```
x = 7
y = 5
x += y # ahora "x" vale 12
x -= 2 # ahora "x" vale 10
```

Este tipo de asignaciones funcionan para todas las operaciones entre números

Reglas de Prioridad

- Se respetan las reglas de prioridad de una ecuación matemática tradicional:
 - Primero se evalúa lo que está entre paréntesis.
 - Segundo multiplicaciones, divisiones y módulos.
 - Por último, sumas y restas.
- Ejemplo:

$$\underbrace{3 * \underbrace{(5 - 2)} / 7 + 12 - 7 * \underbrace{(4 - 1)}}_{\text{Evaluación final}}$$

Operaciones entre Strings

Operaciones entre strings:

Operación	Significado
<code>a + b</code>	Concatenación
<code>a[i]</code>	Indexación
<code>a[i:j]</code>	Cortar
<code>a * b</code>	Repetición (b es entero!)

Ejemplos:

```
s1 = "programacion"
s2 = "2025" # Como tiene comillas es un string
s3 = s1 + s2 # Da como resultado programacion2023
inicial = s3[0] # Devuelve la primera letra
tresletras = s3[2:5] # Devuelve la 3ra, 4ta y 5ta letra
```

Comparaciones lógicas

- Dan como resultado un booleano

Operación	Resultado
5 > 9	False
5 < 9	True
5 >= 9	False
5 <= 9	True
5 == 9	False
5 != 9	True

Funciones

- Es posible realizar tareas más complejas que operaciones matemáticas utilizando **funciones**.
- Para “llamar” a una función es necesario indicar su nombre y luego **siempre** van **paréntesis**.
- Si la función necesita algún dato o variable para trabajar, estos se indican entre los paréntesis.

```
s1 = "programacion 2025"  
# podemos usar print para mostrar un mensaje por pantalla  
print("Ejemplos de impresiones")  
# o para mostrar el contenido de variables  
print(s1)
```

Cambiando entre Tipo de Datos

- Para determinar qué tipo de datos tiene una variable, es posible usar la función **type**.

```
>>> var1 = 7
>>> type(var1)
<class 'int'>
>>> var2 = "7"
>>> type(var2)
<class 'str'>
```

- Es posible convertir una variable a otro tipo de datos, siempre y cuando el contenido de la variable sea compatible con ese tipo de dato.

```
>>> var3 = float(var2)
>>> type(var3)
<class 'float'>
>>> var4 = "abc"
>>> float(var4)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: could not convert string to float: 'abc'
```

Corriendo un código

- Existen 2 formas básicas de correr un programa:
 - Modo interactivo
 - Scripting

Corriendo un código: interactivo

- El modo interactivo se basa completamente en una consola.
- El usuario ingresa sentencias de código y obtiene los resultados de manera visual.

```
>>> a = 32
>>> b = 54
>>> a+b
86
>>> type(a)
<class 'int'>
>>>
```

Corriendo un código: *scripting*

- Un ***script*** es un archivo de texto con sentencias que son ejecutadas en orden por un intérprete.
- El ***script*** termina cuando se ejecutaron todas las líneas o cuando se encontró un error.
- No hay posibilidad de corregir errores durante su ejecución

```
script.py
~/Documents
1 var1 = 123.45
2 var2 = 345.67
3 print (var1 / var2)
4 print (var1 * var2)
```

```
matias@matias-laptop:~/Documents$ python script.py
0.3571325252408366
42672.961500000005
```

Errores

- Durante la ejecución de un código, es muy frecuente encontrar errores.
- Los errores se pueden clasificar en 3 tipos:
 - Compilación
 - Ejecución
 - Lógica
- En los dos primeros casos, el intérprete nos dará mensaje con el error que encontró y en qué línea ocurrió.

Errores: compilación

- Son los errores más comunes y están asociados principalmente a errores de sintaxis.

```
>>> 1a = 1234
      File "<stdin>", line 1
        1a = 1234
          ^
SyntaxError: invalid syntax
>>> s1 = "error
      File "<stdin>", line 1
        s1 = "error
              ^
SyntaxError: EOL while scanning string literal
```

Errores: ejecución

- Estos errores ocurren cuando se pide al intérprete realizar una operación que no puede hacer.

```
>>> n1 = 8
>>> n2 = 0
>>> n1 / n2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
>>> s1 = "errores"
>>> s1[10]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
```


Errores: lógica

- Surgen si hay algún error en el planteo de la solución.
- Usualmente el programa corre sin problemas, pero el resultado obtenido no tiene relación con lo pedido.

```
>>> cat1 = 3
>>> cat2 = 4
>>> (cat1**2 + cat2**2)**1/2
12.5
>>>
>>> (cat1**2 + cat2**2)**0.5
5.0
```